
UNDERSTANDING GRAPH: A RECURSIVE MEDIUM FOR PERSISTENT UNDERSTANDING

A PREPRINT

Henrik Westerberg
Emergent Wisdom
henrik.westerberg@emergentwisdom.org

April 7, 2026
Revised: August 24, 2026

ABSTRACT

AI systems commonly retain source inputs, outputs, and user facts while discarding communicable changes in interpretation between them: questions, tensions, connections, hypotheses, and revisions. We study how to preserve those changes for an agent’s future cognition. We call this *self-directed understanding*: simulating functional features of serious inquiry without claiming human experience or access to hidden computation.

We present the Understanding Graph, an implemented architecture for a continuing Reader. Its persistent graph is the medium through which the Reader works: material shifts become typed and versioned, while competing and displaced views remain inspectable. The Reader may reject state-dependent provocations and re-enter prior state when useful. When a committed update conditions a later encounter and produces another material change, the recursion forms an *understanding spiral*. Encountered passages and produced prose or code share an ordered document surface, linking understanding to exact units of work.

The contribution is a fluid protocol combining self-addressed encounter, persistent interpretive change, agent-chosen re-entry, cross-instance continuity, and graph-native documents. The implementation makes this proposal concrete and inspectable, but does not establish improved understanding or downstream performance. Controlled evaluation must separately test whether repeated re-entry changes the externalized trajectory beyond matched storage and rehearsal, and whether any change improves later work.

Keywords AI Memory, Understanding Graphs, Self-Directed Understanding, Recursive Memory, Interpretive State, Provenance, Graph-Native Artifacts

1 Introduction

The motivating question is deliberately unusual: what if an AI encountered a book, experiment, codebase, or draft not primarily to answer a question for us, but to change what its future instances would be able to understand? A human reader does not merely extract facts. The reader is surprised, unsettled, reminded of earlier experiences, drawn toward unresolved questions, and sometimes changed by the encounter. Much of that movement never appears in a review or final answer. It becomes part of the reader who approaches the next page.

Most language-model memory is organized around a different consumer: the user or the next task. Retrieval-Augmented Generation (RAG) [1] indexes external material so a model can answer later queries; agent memories extract facts, events, reflections, or reusable procedures. These are valuable objectives. They do not, by themselves, require a model to preserve how an encounter altered its interpretive state and then make that alteration part of another encounter. When a session ends, the next instance may recover what was said while losing the trajectory of noticing, doubt, and revision that made the material consequential.

One original requirement came from *Entangled Alignment*’s Missing Reader problem [2]: collections preserve sources and conclusions more readily than the reader-side questions, expectations, surprises, recalled associations, uncertainty, and revisions through which material becomes meaningful. The Understanding Graph was its separate graph-scaffolding hypothesis: make those updates source-bound and addressable across works, retain displaced interpretations, and give a later Reader provenance-bearing state for re-entry and audit. The broader proposal also considered graph-derived training traces and a stable Reader instruction or persona, but those are separable extension hypotheses; neither defines the general Understanding Graph.

When a scientist reads “the results suggest a correlation between X and Y,” the consequential response may be: *How strong? What is the sample size? Could confounders explain this? Does “suggest” mark justified uncertainty?* A conventional note may store the reported correlation. An understanding trace instead records what the sentence did to the Reader’s model of the problem.

Language models create an opportunity, but not the one claimed by a naive reading of chain-of-thought. Their full computation does not “happen in tokens,” and generated explanations need not faithfully reveal the computation that produced them [3, 4]. What tokens provide is a low-friction surface on which part of an encounter can be deliberately externalized. The stored object is therefore neither understanding in its entirety nor a mechanistic transcript. It is a structured, revisable testimony about an interpretive transition.

Definition 1.1 (Self-directed understanding spiral). Given graph state G_k and a concrete purpose q_k , a re-entry operator selects bounded prior state $a_k = A(G_k, q_k)$. A meaningful encounter e_k —a source passage, runtime result, artifact unit, contradiction, or rereading—leads a Reader under protocol p to author an externalized update $u_k = R(e_k, a_k, p)$ and commit $G_{k+1} = C(G_k, u_k)$. The process is *self-directed* when the intended consumer of u_k is the Reader’s own future cognition rather than only an immediate external answer. It becomes an *understanding spiral* when the Reader re-enters G_{k+1} and lets the altered state condition what it notices, questions, connects, or changes next. Storage without re-entry is archival; re-entry without a material update is retrieval, not another turn of the spiral.

“For itself” is operational, not metaphysical. The continuing Reader is a functional role instantiated by a shared protocol, provenance, and graph; it need not be the same model process, and the definition makes no claim of consciousness or uninterrupted personal identity. Self-directed does not mean independent of human direction: a user may set or revise the purpose, inspect the graph, and guide the work, while the future Reader remains a primary intended consumer of committed updates. The design nevertheless tries to *humanize* the interaction in a precise sense: it prompts for features of serious human inquiry that optimized answer production often suppresses—subjective perspective, surprise, hesitation, empathy, unresolved tension, and willingness to revise. A later instance is asked to re-enter this accumulated history before continuing.

We name the architecture the “Understanding Graph” to distinguish its primary unit of storage from a conventional Knowledge Graph. A Knowledge Graph usually represents entities and claims about a domain. The Understanding Graph also represents graph-linked transitions: *I expected this; this evidence disturbed that expectation; an older question now changes what the disturbance means; I remain uncertain here*. It places produced documents and code in the same substrate, allowing an artifact to remain linked to the questions, decisions, and rejected alternatives that shaped it. The graph is thus both an evolving interpretive state and a workspace in which that state can become consequential.

The primitive components—graphs, typed edges, belief revision, provenance, and reflection—are established, and recent graph-memory systems overlap substantially with individual mechanisms. The contribution is their organization around a narrower, testable objective:

1. a graph-canonical medium in which meaningful encounters can become authored updates and later re-entry can recursively change what follows;
2. a typed, chronological record of interpretive change, including uncertainty, alternatives, and revision rather than only stable conclusions;
3. an implemented mechanism for functional continuity through shared graph state that later model instances explicitly re-enter; and
4. linked cognitive and document surfaces, including graph-native artifacts, so understanding can change exact source, prose, or code units and retain inspectable provenance.

This organization addresses the “red thread” problem in long-horizon cognition: retrieval can recover what was discussed yet fail to recover where an interpretation was moving. Re-entry instead places the latest update beside older questions, tensions, alternatives, and evidence. A new relation may then become salient precisely because the graph changed. The resulting second-order update is not guaranteed to be novel, correct, or causally faithful; the architecture makes the proposed movement persistent and inspectable so those properties can be tested.

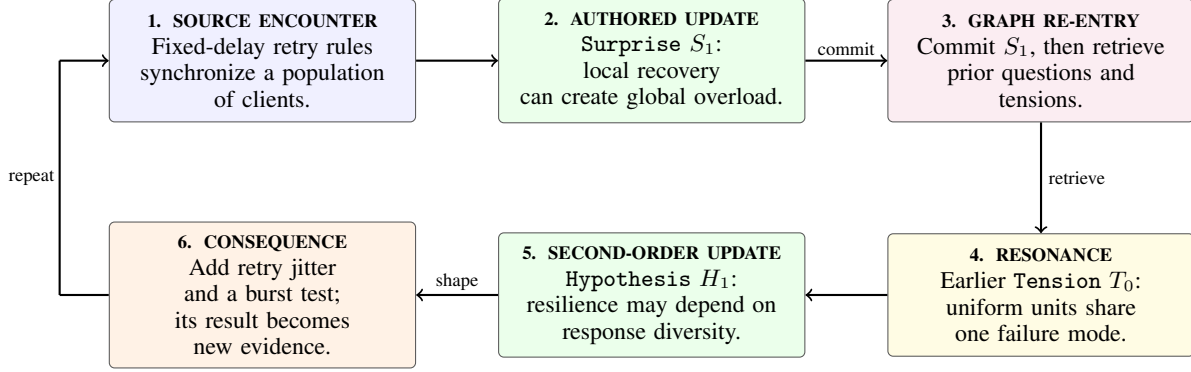


Figure 1: The graph-mediated understanding spiral, shown through an illustrative trace. A source-grounded update is committed and brought into contact with prior graph state. The Reader authors a second-order hypothesis that changes an artifact; evidence from that artifact becomes material for a later encounter. The nodes record revisable externalized testimony, not hidden model computation.

Running example (illustrative). Suppose an earlier encounter with crop monocultures led the Reader to commit a Tension T_0 : uniform units can share a single failure mode. Later, a systems report describes clients that retry after the same fixed delay. At a Thought Moment, the Reader authors a source-linked Surprise S_1 : a recovery rule that protects each client can synchronize the population and intensify overload. The account is linked to the encountered passage through `learned_from`. Here *resonance* means only that a retrieved juxtaposition occasions another material update.

The protocol does not treat S_1 as an endpoint. On graph re-entry, a bounded query resurfaces T_0 . Their resonance prompts a second authored update, Hypothesis H_1 : under common-mode stress, replica count alone is insufficient; resilience may depend on diversity in how replicas respond. Linking H_1 through `abstracts_from` to both S_1 and T_0 preserves why the synthesis arose. The hypothesis then changes an artifact: the Reader adds jitter to the retry function and a correlated-burst test. The test result becomes a later encounter. If jitter dampens synchronization but sustained overload still causes collapse, a new Evaluation can narrow, challenge, or supersede H_1 . Each pass therefore begins from a graph enlarged or revised by the previous pass rather than merely repeating a transient chain of thought.

2 Related Work and Conceptual Boundary

Existing memory systems span several overlapping objectives. Extraction systems and virtual-context managers such as Zep/Graphiti [5] and MemGPT [6] preserve facts, events, and conversational state for later use. Indexing systems such as GraphRAG [7] and HippoRAG [8] transform documents into navigable external knowledge. These systems primarily optimize later retrieval or task performance rather than open-ended change in the Reader’s interpretive state.

A growing body of work blurs this boundary. Recent work has refined retrieval through hierarchical summarization (RAPTOR [9]), self-reflection (Self-RAG [10]), corrective mechanisms (CRAG [11]), and hybrid structurization (StructRAG [12]). While Graph of Thoughts [13] structures reasoning only ephemerally during a single inference, our system persists this structure across sessions. Others like Think-on-Graph [14] and RETRO [15] push the boundaries of static knowledge access, yet most remain focused on accessing existing information rather than capturing new understanding.

A second lineage persists reflection and evolving agent experience. Generative Agents stores observations, reflections, and plans and retrieves them to shape later behavior [16]. Reflexion [17] persists verbal lessons across task episodes, while Buffer of Thoughts [18] distills reusable problem-solving templates. A-MEM [19] creates interconnected notes; Hindsight [20] separates world facts, agent experiences, synthesized entity summaries, and evolving beliefs, then recalls and reflects across them. These systems make the simple claim that “other memories discard reasoning” untenable. The relevant distinction is instead what the stored reflection is *for*: task improvement, behavioral simulation, factual continuity, or the Reader’s own open-ended comprehension.

Graph-native agent memory is now a substantial category in its own right [21]. The closest comparator is Kumiho [22], which represents versioned cognitive memory as a graph with typed dependencies, immutable revision history, and references to external work products. MRAgent [23] actively reconstructs task-relevant graph context rather than merely retrieving fixed records. Kumiho’s public version appeared in March 2026, before this paper’s April 7 initial

release; MRAgent’s public arXiv version followed in June 2026 and is included in the present August revision. These systems overlap with the Understanding Graph in graph-native cognition, revision, provenance, iterative access, and in Kumiho’s case links to artifacts. We therefore do not claim priority for those ingredients individually.

The narrower proposal is the recursive process in Figure 1. The Reader preserves a material interpretive update, deliberately re-enters the changed graph, and allows the relation between new and prior state to condition another encounter. The same substrate holds produced text or code and the cognitive nodes that shaped it. This is best understood as a distinctive design objective and integration, not a categorical boundary around a wholly new species of memory. Individual mechanisms have close ancestors; the empirical question is whether organizing them around recursive, self-directed understanding produces better continuation than a summary, extracted facts, an unstructured notebook, or an equal-budget linear transcript.

The design also belongs to the lineage of extended and distributed cognition. The Extended Mind thesis [34] and cognitive architectures for language agents [35] treat external structure as part of a cognitive process; Luhmann’s communicating Zettelkasten [36], Quillian’s semantic memory [37], and A-MEM [19] each emphasize connected, evolving memory over flat storage. Burgess’s Semantic Spacetime framework [38] similarly foregrounds typed associations, scale separation, identity, and trajectories, while SSTorytime [39] applies that lineage to process-shaped story maps. The Understanding Graph arrives from the LLM side, but shares the premise that memory becomes cognitively interesting when it preserves typed movement through relations rather than facts alone.

3 Recursive Understanding Architecture

We employ a “Decoupled State” architecture in which the *Graph* stores externalized interpretive state and artifact provenance, distinct from the *LLM* that renders or extends it. The essential mechanism is not storage alone but a recursive possibility: the model can encounter material, author a substantive update, commit it, and later re-enter an altered structure that changes what follows. Typed representation makes the movement addressable; revision preserves change without erasing history; retrieval determines which prior structures exert pressure on a later pass.

3.1 The Continuing Reader Protocol

The graph alone does not create self-directed understanding. The Reader protocol instead establishes a coached medium: while the protocol is active, substantive artifact state and every communicable, material update that could matter to the work or the Reader’s future inquiry are graph-canonical, while chat mirrors status, questions, and committed results. This *medium-integrity invariant* is stronger than occasional note-taking but does not prescribe a state machine. It is a protocol commitment, not a property the runtime can verify (Section 4.2). The Reader may orient, preserve, search, make, test, connect, disrupt, re-enter, reconsider, or pause in whatever sequence the work demands.

The Reader advances through the work’s native form and notices *Thought Moments*: encounters at which an expectation breaks, a tension appears, an alternative becomes plausible, a question becomes urgent, a test changes the model, or the stakes acquire human weight. At each such moment it asks what substantively changed and what now deserves different attention. The account is preserved as the change emerges, rather than reconstructed only as a polished rationale at the end. Re-entry is available whenever prior graph state may alter the next move; it is central to the spiral hypothesis, not a ritual required after every call.

Within one continuing Reader’s project, the protocol is deliberately singular across works. Starting a new source does not create an empty Reader with a document-local notebook: the Reader role, orientation instructions, project graph, and provenance history persist. The implementation may invoke a new model process, but that process is asked to continue the same graph-mediated Reader. Material from one work can therefore supply active questions and concepts when the Reader later encounters another, subject to context selection and the model’s ability to use the retrieved state.

This is a persistence mechanism for an *understanding process*, not merely a note format. A summary tells another party what a source contains. A task trace records steps toward a specified answer. The Reader’s update is addressed forward to its own continuation: it preserves what the encounter changed, including uncertainty that has not yet become a conclusion. The distinction is an optimization target rather than a vocabulary test; a summary can contain insight, and a purported understanding trace can collapse into summary. Repeated graph use makes the difference testable: did the committed state alter a later question, relation, revision, or artifact decision?

We call a change worth preserving a *material, articulable update*: a question, relation, revision, or consequence that could alter later attention or work. The protocol asks for every such update, not every token-level step. Completeness is unknowable, and node quotas would reward decorative introspection. A pass can stop when it yields no material

update, when a proposed update merely paraphrases existing state, or when its declared attention budget is exhausted. An honest no-update outcome is preferable to manufacturing depth.

First-person voice remains a deliberate variable. It asks the model to preserve “maybe,” situated stakes, and emotional or ethical salience rather than defaulting to authoritative neutrality. This does not establish phenomenal feeling. The success criterion is behavioral: later instances should recover and appropriately revise the earlier trajectory, and produced work should be measurably informed by graph-carried state.

Chronological document understanding. A minimal user-facing application of the protocol is a chronological document reader. In its no-lookahead form, a host stages a book, paper, transcript, or other document without exposing its body to the model. Rather than placing the entire source in the model’s immediate context, the system reveals successive bounded passages in source order, favoring coherent boundaries where the source permits. Each encountered passage becomes an exact, addressable document node. When an encounter produces a question, expectation, association, tension, or revision worth carrying forward, the Reader may attach ordinary passage-grounded testimony before continuing. The next passage therefore meets a graph shaped by what was actually encountered, not a retrospective account produced after seeing the whole document.

During or after the reading, a user can ask open-ended questions against the continuing graph. Re-entry can place exact passages beside provisional interpretations, unresolved alternatives, and their recorded provenance, while allowing the Reader to revise or reject its earlier account. Because the cursor and graph persist, reading and questioning can continue across model sessions without reloading the entire source. This ordinary product form is distinct from CMP: it creates neither reserved Thinking blocks nor a synthetic training corpus. The resulting graph is an inspectable continuity mechanism, not evidence that the model achieved “true understanding” or outperformed other reading and memory methods.

Adaptive codebase understanding. A complementary application pattern is adaptive codebase understanding. Code has no single analogue of page order, so a Reader can encounter bounded, addressable units—entry points, types, functions, tests, configuration, and runtime observations—in a provisional, purpose-relative order informed by dependency, behavior, and conceptual relation. Source-linked questions, models, tensions, and revisions from each encounter can then condition which unit is examined next; the traversal may branch, revisit earlier units, or stop without claiming exhaustive coverage. This preserves an inspectable account of how an architectural or behavioral model developed, while leaving open whether staged, graph-conditioned traversal improves comprehension or coding performance over matched alternatives. The current implementation provides graph-native code units, coding-shaped re-entry, and runtime evidence, but not an automatic repository traversal scheduler; this application remains proposed.

3.2 Repeated Re-entry and Context Selection

The spiral depends on selecting enough prior state to change the next act without turning the full graph into an indiscriminate prompt. Re-entry is therefore purpose-framed and bounded. It is not limited to startup: the Reader may re-enter after a meaningful source, test, artifact unit, contradiction, decision, or rereading, while allowing a no-update outcome when the packet exposes nothing material.

3.2.1 Purpose-Framed Re-entry

Purpose-framed re-entry uses two orthogonal axes. A workflow identifies the domain of work and its native evidence; a stance selects the kind of epistemic pressure desired, such as deepening, resistance, connection, disruption, revisiting, or testing. These are retrieval choices rather than capability boundaries. Instead of returning a flat similarity list, the packet separates a provisional *baseline* from *resistance* and *evidence*, retaining typed relations and relevant revisions.

The packet is workflow-sensitive. Reading and research can retrieve bounded passages across a document tree and report uncovered query terms as possible insufficiency rather than licensing a guess. Coding can surface current artifact units with prior revisions and runtime evidence. Any workflow can seek connective or disruptive pressure when local relevance has become too narrow. In writing, if graph material changes a unit, *inspired_by* records the author’s claim of influence; if nothing catches, no connection is required. Structural imbalance can be reported without becoming a ratio target.

The returned graph is provisional testimony, not authority. The model is asked to let its unresolved momentum alter what it notices or tries, then continue in the task’s native form. A useful re-entry can confirm the current direction, expose resistance, recover a forgotten question, or reveal that the current packet is insufficient. The critical behavioral claim is not that retrieval itself creates insight, but that the changed graph supplies material that can make a later update possible and inspectable.

3.2.2 State-Dependent Choice and Anti-Locality

The design begins from a practical hypothesis: answer-oriented interaction often favors closure, whereas an unbounded instruction to “think divergently” supplies neither a concrete object nor a stopping condition. The graph is intended to change the cost of a tangent. A bounded provocation can be judged against the live purpose, and any material update can accrue as inspectable, revisable structure alongside the requested work.

When optional suggestion guidance is enabled, the coaching contract can offer a small weighted set of concrete multi-step routes at the start of substantive work and at natural choice points. The aid is intended to deepen or diversify understanding, recover neglected material, test the current view, or make a useful connection newly visible. Route weights reflect the task, workflow, current topology, unresolved material, and recently rolled action kinds. Each route carries a rationale and a stance for a possible re-entry packet. Higher weights deserve stronger consideration, but the Reader may choose, combine, modify, reject, replace, or skip the routes; the mechanism remains guidance rather than a governor and provides no guarantee of better understanding. In direct mode the Reader chooses these moves without ambient suggestions, while retaining the same graph capabilities and the ability to request a roll on demand. This chooser is one implementation of a choice point, not a necessary condition of an understanding spiral. Recent-action down-weighting is short-lived process memory rather than learned strategy persisted in the graph.

Repeated re-entry can still remain trapped in familiar regions. A rolled route may therefore compare distant nodes, sample cold material, seek a grounded bridge, or temporarily force a bisociation—the intersection of different matrices of thought [29, 30]. “No connection” is a valid result, while a temporary what-if must later survive scrutiny. If an unexpected bridge changes subsequent work, it can be preserved as a source-linked Serendipity, Hypothesis, or other type that states what the result actually became. None of these routes commits a relation automatically.

3.3 Node and Edge Taxonomy

Typed cognitive nodes make different kinds of interpretive movement separately addressable instead of collapsing them into generic “concepts.” Extending the IBIS argumentation lineage [24], the current system defines seventeen ordinary TriggerTypes in five families and one reserved synthetic type (Table 1). This taxonomy describes cognitive testimony. Document nodes share the underlying node record but are distinguished by document fields and ordered topology; they are not cognitive testimony merely because they retain trigger metadata. The list is an implemented design vocabulary, not a formally minimal ontology.

Family	Types	Function in the externalized account
Core Reasoning	foundation, surprise, repetition consequence, tension, question, hypothesis	Priors, expectation violations, and recurring patterns Causal implications, conflicts, unknowns, and provisional explanations
Discovery Meta	serendipity, decision, experiment analysis, model, randomness, reference, library	Unexpected bridges, choices, and validation attempts Consolidation, abstraction, exploration, and source organization
Process Reserved	prediction, evaluation thinking	Forward commitments and judgments Extension-only synthetic Reader blocks; unavailable to ordinary work

Table 1: Implemented node families. A type records what an authored update became; it does not prove that the update was complete, sincere, or causally load-bearing.

Edges state how cognitive updates and document units relate. The validator accepts sixteen specific types plus relates as a tracked fallback. Epistemic relations include `learned_from`, `questions`, `answers`, `contradicts`, `validates`, `invalidates`, `refines`, `contextualizes`, `abstracts_from`, `diverse_from`, and `supersedes`; `contains` and `next` encode document topology. The distinctions preserve plural state: `diverse_from` keeps multiple perspectives live simultaneously without collapsing them into one view, `contradicts` records explicit conflict, and `supersedes` retires a displaced commitment. Cross-surface provenance distinguishes `implements` (abstract cognitive commitment $\rightarrow$ produced unit), `expresses` (produced unit $\rightarrow$ concept), and `inspired_by` (produced unit $\rightarrow$ graph material reported as influential). The epistemic `learned_from` relation instead points from a cognitive update to the source or artifact encounter identified as its evidence. Prompt guidance favors the most specific defensible relation, while diagnostics make fallback overuse visible without turning edge diversity into a quota.

3.4 Cognitive and Document Surfaces

Traditional systems separate files from knowledge graphs, but the Understanding Graph places cognitive testimony and addressable documents in one substrate while preserving their different roles. The *cognitive surface* holds questions, tensions, alternatives, decisions, models, and material updates. The *document surface* holds ordered semantic units from both encountered material and produced work: source passages, document sections, prose passages, functions, classes, types, tests, import groups, and other coherent units. A source passage and an authored function can therefore both be document nodes, while only the latter is an artifact unit. The cognitive surface may be used without producing an artifact; artifact production is a possible consequence of understanding, not its required endpoint.

A document node is not merely a retrieval chunk. It is a stable address for one semantic unit of work. Incoming text and produced prose or code can both be divided at independently meaningful boundaries—a source passage, argument move, function, class, type, or test—while a rendered page or generated file remains a projection of the ordered tree. The nodes carry content and hierarchy and are linked by *contains* and *next*. In graph-native coding these ordered nodes are canonical source and generated files are executable projections. In writing, chapters and scenes can be containers while beat-, image-, exchange-, paragraph-, or sentence-sized leaves remain independently movable and revisable. Granularity follows one addressable center of attention, not a word or node quota.

Figure 2 renders the running example as stored relations. The encountered passage and produced code are both document nodes, while the surprise, tension, and hypothesis remain cognitive testimony; typed relations connect the surfaces without collapsing them.

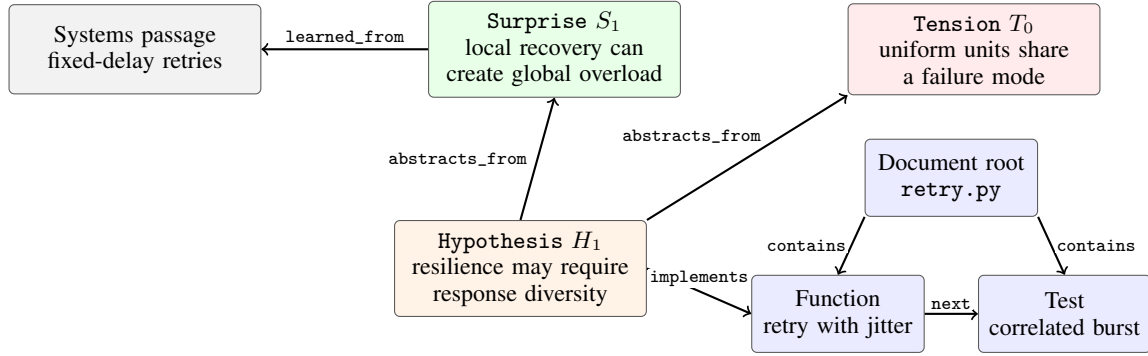


Figure 2: Typed topology for the illustrative spiral in Figure 1. The encountered passage and the produced `retry.py` tree occupy the document surface; the surprise, tension, and hypothesis occupy the cognitive surface. The hypothesis points to the exact function that implements it, while *contains* and *next* make the separately addressable function and test an ordered document tree. These links preserve recorded, unit-level provenance.

The Reader can attach authored testimony to the exact source passage or artifact unit that occasioned it; the note may remain open or later be marked resolved without being erased. Source nodes and *learned_from* edges preserve stated encounter provenance, while semantic edges place the update within prior understanding. Source-free deliberation is also possible: a contradiction, experiment, or decision can become an encounter when it materially changes future attention.

These typed directions support bidirectional questions: which functions implement this decision, and why does this exact passage, function, or test exist? The answer can combine the unit’s recorded purpose, its origin commit, local relation rationales, and revision history. It is a precise account of what was recorded, not proof of a complete or psychologically causal account; absence of a link means that no rationale was recorded, not that no cause existed.

Creation, revision, splitting, movement, and merging are atomic graph mutations; generation is a post-commit projection from canonical graph state. Moving a function between modules preserves its identity, history, and relations; merging redirects external semantic relations to the surviving unit. Splitting cannot infer which child inherits a unit-local rationale, so relations on the emptied container must be reviewed and reattached only where they remain true. Typed testimony can thus stay linked to the exact unit it concerns, preserving a contemporaneous account rather than requiring wholly post-hoc rationale. After generation, testing, or rereading, the artifact becomes another encounter: its behavior can trigger a new evaluation, contradiction, or revision and return the spiral to the cognitive surface.

The topology also bounds context in large documents. Agents navigate the tree and load selected units for the current purpose, then regenerate external projections from graph state. Whether this improves scaling or task quality over ordinary files remains to be benchmarked.

3.5 Revision, Accretion, and Consolidation

The closest historical analogy is the laboratory notebook. Published science foregrounds results—the successful experiment, the proven theorem—while the notebook can retain failed experiments, crossed-out hypotheses, margin notes, and the gradual narrowing from confusion to clarity. Researchers can then build on parts of each other’s *process*, not just conclusions. The Understanding Graph is an LLM-compatible version of that intuition: a notebook whose entries are typed, linked to their sources, and directly available to the next Reader.

Concretely, memory can record correction as well as accumulation (Table 2). When the Reader refines an interpretation, it can create a *supersedes* edge from the new node to the displaced one rather than deleting history. This belongs to the lineage of Truth Maintenance Systems [25] and overlaps with versioned graph memories such as Graphiti [5] and Kumiho [22]; AriGraph [26] likewise integrates episodic and semantic memory into a graph world model. Every edge carries a natural-language *why*, so the transition is not merely timestamped.

Ordinary mutations sent through `graph_batch` are grouped into atomic commits carrying a timestamp, required client-declared `agent_name`, optional human author, message, and affected node and edge identifiers. Failed batches roll back rather than leaving partial topology. When a future Reader retrieves a node, the commit history can expose the stated purpose that accompanied its creation or revision. The current implementation does not provide named branches or full committed-state revert.

Persistence is not epistemic endorsement. Durable nodes may be candidates, objections, alternatives, or provisional syntheses. A *Decision* or *Analysis* can state what presently governs; *diverse_from*, contextualization, and supersession preserve plurality and change. Because no universal candidate-status field exists, consumers must interpret type, relations, content, and protocol rather than treat every commit as settled belief.

Accretion and correction. In problem-solving, supersession can record repair of a flawed hypothesis or obsolete design. In reading, the graph may be more additive: a new interpretation can link through *learned_from* without invalidating earlier context. Workflow-specific prompts make this distinction explicit, allowing both iteration of a solution and accretion of a reading. Whether the resulting topologies differ reliably is an empirical question.

Signal	Epistemic Function	Use Case
<code>learned_from</code>	<i>Accretion</i> : Records an additional interpretation and its stated source. (“How did this arise?”)	Reading, History
<code>supersedes</code>	<i>Correction</i> : Marks one commitment as displaced by a newer governing commitment. (“What governs now?”)	Coding, Architecture
<code>validates</code>	<i>Confirmation</i> : Records a result presented as support for a hypothesis. (“What supports this?”)	Science, Fact-Checking

Table 2: Three update signals distinguish accumulation, correction, and confirmation.

Consolidation without erasure. A major challenge is compressing a complex region without losing its unresolved structure. An *Analysis* node is intended as an active, revisable synthesis rather than a passive summary cache: it can record a selected view while preserving alternatives linked through *diverse_from*. Linking the analysis to the testimony it integrates makes its stated provenance inspectable. In this limited sense it acts as a cognitive ratchet, but dependency invalidation is not automatic; if underlying nodes change without later revision, the analysis can become stale like any other derived representation.

Supersession semantics. Let $G = (N, E)$ and define $n_1 \succ n_2$ when E contains $(n_1, n_2, \text{supersedes})$. If $\ell(n) \in \{0, 1\}$ is the stored live-state predicate and $\succ$ is acyclic, the currently undisplaced set is

$$N_A = \{n \in N \mid \ell(n) = 1 \wedge \nexists n' : n' \succ n\}.$$

Branching may leave multiple current successors, while the transitive closure $\succ^*$ preserves recorded history. The dedicated `graph_supersede` operation atomically preserves and retires the displaced node; generic creation or mutation of a *supersedes* edge is rejected. Global acyclicity is nevertheless an intended lifecycle invariant rather than a property proved for every writable history. A `Redis`→`Postgres`→`event-log` chain can therefore record successive governing architectures while only the latest remains active. Each transition carries an authored *why*, creating a semantic changelog whose claims are inspectable but not mechanically verified. This is inspired by the bookkeeping problem in AGM belief revision [27], but it is not an AGM operator and does not enforce deductive closure, minimal change, or consistency restoration. Nor does explicit supersession solve the behavioral difficulty LLMs can have revising conclusions under counterevidence [28]; it makes a proposed revision persistent and challengeable.

4 Implementation and Enforcement Status

The Understanding Graph is implemented as a standalone server exposing graph operations via a standardized protocol. This design decouples the graph engine from any specific LLM provider, enabling experimentation with different models. The complete implementation—including the graph engine, MCP server, React-based visualization frontend, and paper source—is available as open source at <https://github.com/emergent-wisdom/understanding-graph>. Multiple SQLite project stores can retain explicit cross-project references without merging their contents.

Unlike a transient RAG session, the system produces persistent artifacts: graph snapshots of evolving externalized state; semantic changelogs recording what changed and the stated reason; and graph-native documents whose units remain linked to testimony through relations such as `implements`, `expresses`, `inspired_by`, and `learned_from`. Reserved synthetic blocks belong only to the isolated Reader extension described in Section 5.

4.1 Technical Foundation

The system relies on SQLite as its primary storage backend, chosen for single-file portability and ACID transaction guarantees. Each project maintains a dedicated `store.db` file containing tables for nodes, edges, commits, and `text_sources`, with flexible JSON metadata fields to support extensibility.

Interaction is mediated through the Model Context Protocol (MCP) [31]. The server exposes eight enforced `TOOL_MODE` capability registries: general (the default), reading, research, coding, collaborative coding, writing, full, and synthetic Reader. The tool surface covers purpose-framed re-entry, graph mutation and query, source reading, graph-native artifacts, coordination, and optional exploration. Successful atomic batches return a bounded focus set for the next re-entry. A trusted hosted gateway can supply authenticated human authorship; the standalone engine does not establish that a client-supplied identity is authentic.

The current implementation realizes chronological document reading through `source_load`, which stores the source while its model-facing response returns metadata rather than the source body; when the source is supplied by a host-controlled file path rather than inline text, the Reader need not encounter it in advance. Successive `source_read` calls reveal and atomically commit only the next bounded span as an ordered document node. Between encounters, the Reader may attach passage-grounded testimony using ordinary cognitive types, or add none when no material update has emerged. Here `workflow=reading` selects a reading-shaped re-entry packet, whereas startup `TOOL_MODE=reading` only narrows the callable surface; neither exposes reserved Thinking blocks, which belong exclusively to the optional `synthetic_reader` CMP extension in Section 5. These runtime contracts enforce visibility and persisted order, not faithful attention or freedom from prior knowledge.

The `graph_understand` operation realizes purpose-framed re-entry through an explicit work-domain workflow and orthogonal epistemic stance. It uses embeddings when available and falls back deterministically to lexical relevance; after mutation, `graph_batch` can return exact focus-node identifiers so changed topology does not compete only on similarity. The optional weighted chooser is exposed as `graph_suggest_next`; the server-session setting `UG_GUIDANCE_MODE` selects ambient guided prompts or direct use without removing the endpoint. Grounded bridge search and stochastic or forced bisociation are exposed through `graph_discover_ground` and `graph_random`. A separate `graph_thermostat` reports a coarse deepen/connect/disrupt compatibility pulse. These operations propose routes but do not commit semantic relations automatically.

For graphs above 50 nodes, `graph_context` automatically compacts its response, while `graph_skeleton` provides a structural digest of counts, large communities, hubs, and recent activity. Selected regions can then be paged through `graph_context_region`. Across sessions, `graph_updates` retrieves bounded topological changes since a timestamp rather than reloading the full graph. These are implementation strategies for navigation and synchronization; purpose-framed re-entry still determines which state matters to the current work.

To support semantic operations, nodes can optionally store vector embeddings. The main batch-creation path attempts embedding generation when a provider is available, while failures are non-fatal; at startup the server requests a declinable background warmup. Availability nevertheless depends on completed model loading and embedding coverage, so early calls and older unembedded nodes may receive only lexical protection. Existing nodes can be filled later through `graph_backfill_embeddings`, while `graph_practice` and graph statistics report actual coverage. Completed embeddings enable cosine similarity search with configurable thresholds and semantic gap detection between unconnected but conceptually similar nodes.

The accompanying 3D frontend provides a read-only window onto externalized graph activity. Observers can watch nodes appear, inspect typed relations and commit authorship, and reveal superseded nodes as a historical “ghost” of prior accounts. This makes the persistent workspace observable.

4.2 Enforcement Boundary

Runtime capability access is enforced through `TOOL_MODE` allow-lists and visibility projection: the MCP server removes reserved `Thinking` nodes, their incident edges, histories, document paths, and search effects from every ordinary mode. Startup instructions and tool-return prompts separately coach the recursive practice; they are guidance within the hard capability boundary, not a second enforcement layer.

The server rejects malformed or unauthorized operations—including invalid triggers and edges, self-loops, broken document topology, forbidden cross-mode access, ungrounded synthetic blocks, and batches that would leave newly created cognitive nodes structurally orphaned. It cannot determine whether a formally valid note reflects honest attention, whether an edge rationale is insightful, whether repeated re-entry was causally useful, or whether the graph is complete. Those higher-order judgments remain coached, diagnostic, or empirical.

Table 3 makes this boundary explicit.

Tier	Current examples	What it does not establish
Runtime-enforced	<code>TOOL_MODE</code> allow-lists; reserved-block visibility; typed batch transactions; strict document topology	Semantic quality, completeness, or causal faithfulness
Coached/advisory	Medium integrity; Thought-Moment timing; agent-chosen moves; weighted suggestions; repeated re-entry; relation choice; structural diagnostics	Compliance by an agent that ignores the guidance
Proposed/unevaluated	Automatic semantic verifier; adaptive code-base traversal; default spiral benefit; training on synthetic Reader blocks	Any improvement in understanding, safety, or downstream performance

Table 3: Implementation status. Runtime validity, prompted practice, and empirical hypotheses are distinct layers.

5 Optional Extensions

The recursive contract does not require a particular agent roster. In multi-agent use, the graph can serve as a stigmergic medium: heterogeneous agents leave source-linked updates that others validate, challenge, or refine without relying only on direct messages. Collaborative-coding mode adds a persistent task queue, handoffs, and advisory subtree locks. This can preserve disagreement as first-class topology, but also adds contention, redundant work, and integration cost; a shared graph does not guarantee that more agents help.

Within *Entangled Alignment*, the graph also served as infrastructure for Chronological Metacognitive Pretraining (CMP) [2]. Three possible roles remain distinct: a Teacher-side scaffold, a serialization and audit surface for candidate training data, and an optional live deployment graph. CMP is a proposed reuse of source-adjacent testimony, not the definition of understanding. A dedicated `synthetic_reader` mode can reconstruct chronological training blocks grounded in selected graph nodes. Only that mode can read or write reserved `Thinking` nodes; ordinary workflows see a projection that excludes them and their incident relations. Append-only operations preserve committed order, but part of the stop-and-commit rhythm remains prompt-governed, the validator checks only the opening anchor of the Reader Core—that proposal’s longer stable Reader instruction and persona—and no downstream model has yet been trained on the corpus. Chronology and provenance are safeguards, not evidence of training utility or trace faithfulness.

Optional coaching can also perturb or evaluate a synthesis. PURE [33] asks whether a proposal is parsimonious, unique, realizable, and expansive; stronger bisociation variants deliberately force distant concepts into contact. Temporal attention rankings separately expose nodes that are recently or frequently accessed, but decay is queried on demand rather than enforced in every retrieval. These are bypassable, unvalidated research instruments. They are extensions of the substrate, not premises of the core claim.

6 Evaluation Framework

The central hypothesis in Definition 1.1 is not that a graph can store many reflective notes. It is that committing an update and then re-entering the changed structure can alter what a later pass notices, connects, or revises. The primary experiment should therefore be an *iterative re-entry ablation*. Using the same model snapshot, source sequence, total token budget, and final task, compare zero, one, and multiple re-entry opportunities. Include an *archival graph* control in which the model writes the same kind of nodes but receives no graph state back between encounters; this separates externalization and storage from recursion. A no-memory baseline receives neither persistent state nor graph-writing work.

Where cost permits, cross representation with pass count: typed graph, length-matched summary, extracted facts, persisted linear transcript, and unstructured notebook, each reread zero, once, or repeatedly. Non-graph controls must receive the same number of forward passes and equal opportunities to read and write; otherwise extra inference or rehearsal alone could explain an apparent graph benefit. A smaller preregistered study can select a subset of this factorial design, but should retain the archival control and at least one repeatedly reread non-graph representation.

The chronological-reader application also admits a direct product-level comparison. On documents that fit the whole-context condition, staged no-lookahead reading should be compared with immediate whole-document prompting and a length-matched sequential transcript, holding the source, model snapshot, total token budget, and final questions fixed. Evaluation should separately score withheld-question accuracy, calibration, exact passage support, appropriate revision after later evidence, and continuity when a fresh model instance resumes the reading. This tests whether persistent chronological encounter contributes beyond added inference, source segmentation, or context amortization.

An adaptive codebase-reader study should compare graph-conditioned traversal with static dependency-order traversal, task-matched repository retrieval, and whole-repository prompting where feasible, matching the model, token and tool budget, and available runtime evidence. Because a traversal may benefit merely from exposing better code, the same encountered-unit sequence should also be replayed with archival, no-re-entry state. Architecture questions or change requests withheld until after traversal can then test source-supported architectural accuracy, impact prediction, defect localization, test performance, and continuation by a fresh model instance.

The stochastic chooser introduces another possible cause. Studies using `graph_suggest_next` should compare it with suggestions disabled and with length-matched concrete provocations that are not computed from graph state. Every condition should receive the same number of choice-point calls and forward passes; rolled options, weights, recent-action state, and random seeds should be retained by an instrumented evaluation harness. The current public chooser uses unseeded randomness and does not itself expose a replay seed, so exact replay requires additional instrumentation or future seed support. Otherwise diversity or extra prompting, rather than graph-mediated recursion, could explain an apparent gain.

A graph-mechanism study should also include *query theater*: the same visible calls, prompts, and token budget, but retrieved state is hidden, shuffled, or nonpersistent. This separates graph effects from querying ritual. For the alignment application, Teacher-side construction, Student-side serialization, and live deployment access require separate tests. A transfer experiment can withhold the live graph; a deployment experiment should distinguish grounded hit, grounded absence, and an adversarially injected node. Evidence for one layer does not validate the others.

The running example gives the tests a concrete prediction. After the retry observation produces S_1 , a successful graph condition should be more likely than matched controls to retrieve the older monoculture tension T_0 , form and appropriately qualify H_1 , and change the retry artifact or its tests. Merely restating S_1 is not another turn of the spiral.

The *mechanism hypothesis* asks whether repeated re-entry changes the Reader’s externalized trajectory. Its primary endpoint should be blinded judgment of source-grounded, non-redundant second-order updates that appropriately use earlier state: new connections, qualified hypotheses, revision after counterevidence, contradiction discovery, delayed cross-source transfer, or changes to exact artifact units. Studies should separately report repetition, unsupported-association, and decorative-note rates, because indiscriminate node creation can imitate movement without adding useful understanding. Instrumented traces can establish that a recorded update followed retrieval, but provenance alone does not prove that the retrieved node caused it.

The *utility hypothesis* asks whether any changed trajectory improves later work. Its primary endpoint should be blinded performance on withheld questions or changed requirements that were unknown when memory was constructed, with task-specific measures of artifact correctness, quality, adaptation cost, or transfer where appropriate. A mechanism effect without a utility effect would show altered externalized cognition, not improved work; a utility effect without the predicted process would support a memory benefit, not the proposed spiral mechanism. The paper’s central recursive claim requires evidence for the mechanism hypothesis; stronger claims of practical benefit require evidence for both.

Four secondary comparisons isolate adjacent claims. A *self-addressing ablation* compares the Reader protocol against matched summarization and task-solving prompts, scoring source-grounded changes in expectation, uncertainty, connection, and revision rather than first-person style. A *continuation test* gives a later model instance memory produced before the evaluation questions were known, then measures whether it uses and appropriately revises earlier questions and models. An *artifact-provenance test* changes requirements after code or prose has been authored and compares graph-linked units against ordinary files plus commit messages. A *voice control* preserves semantic content while varying first-person and affective framing, separating functional continuity from merely sounding reflective or human.

The ordinary `graph_score` can support these experiments by checking backward references, supersession, question resolution, edge specificity, connectivity, and rationale quality. It is a structural diagnostic, analogous to branch coverage, not a measure of semantic correctness. A high-scoring graph can be nonsense and a sparse but useful graph can score poorly.

A separate conduct report, `graph_practice`, addresses the complementary blindness: a well-formed graph nobody re-enters is structurally indistinguishable from one that changes what its author does next. It reports how the graph was *worked* rather than how it is shaped—writes per re-entry, the share of predictions that later received a verdict, attribution drift between early and recent commits, refusal rates on the write path, the share of edges that overturn earlier claims, fallback-edge share, and standing decisions that no artifact unit expresses. Each figure names its basis and is stated as a proxy with a hedged reading; where a reading has two bad directions, both are named, since a reading that names only one is an instruction to move the other way without limit. Absence is reported as absence rather than zero, and there is deliberately no aggregate: a single score becomes a target. These are process measures in the sense above. They can support the secondary endpoints without establishing quality—a diligent-looking conduct report can accompany poor understanding, and the readings say so.

Controlled studies should preregister paired materials, model snapshots, token allocation, stopping rules, randomization, blinded judging, contamination probes, and effect sizes. They should retain failed passes and null results rather than selecting only attractive spirals. Runtime enforcement and structural diagnostics are engineering verification, not substitutes for these controls.

7 Limitations and Responsible Interpretation

The first limitation is epistemic. The graph captures an *externalized testimony* authored while a model interprets; it does not expose the computation that produced the model’s output. Chain-of-thought research shows that stated reasoning and causally operative computation can diverge [3, 4]. A provenance edge establishes what the system recorded and where the account points, not that the linked source was uniquely causal. Supersession chains should therefore be read as inspectable claims about interpretive change, not as ground truth about model weights or hidden cognition.

Second, no runtime can verify that the record is complete or honest. Some material changes will remain unarticulated, while a compliant-looking model can manufacture polished testimony after the fact. Structural diagnostics can reveal extreme patterns but may themselves induce compliance behavior; none can determine the right granularity or sincerity of a note.

Third, recursion can amplify error as well as insight. A strongly connected early belief may dominate retrieval, cause repeated exposure, and become resistant to supersession. As a loose analogy, this resembles canalization in predictive-processing accounts [32], although here the groove is explicit topology rather than implicit precision weighting. Ranking and temporal decay can reduce stale context, while grounded exploration can expose distant regions, but neither guarantees correction. Evaluation must look for echoing, premature convergence, and suppressed alternatives, not only for additional connections.

Fourth, representation and retrieval impose their own biases. Typed graphs privilege relations that the schema and model can name; a generic edge can hide important distinctions, while excessive specificity can fragment equivalent ideas. Purpose-framed retrieval is bounded and may omit a decisive but lexically or semantically distant premise. Different tasks may benefit from different structures [12]; the same topology need not be optimal for reading, research, code, and narrative work. A null retrieval result is therefore evidence about the current index and query, not proof that the graph contains no relevant material.

Fifth, the current engine is a portable SQLite implementation with in-memory caches rather than a large-scale graph service. Dense regions increase traversal and context-selection costs, and artifact trees can grow much faster than cognitive testimony. Separate project files provide isolation and can be served independently, while writes within a worker are serialized. The paper does not claim automatic horizontal scaling, live migration, or fault-tolerant dis-

tributed writes. Approximate indices, colder storage, and alternative graph backends may become necessary at larger scale; their effect on provenance and reproducibility would need separate study.

Finally, persistent testimony can be sensitive. It may retain abandoned beliefs, affective reactions, proprietary code, source excerpts, and attribution across revisions. Source and graph text can also contain adversarial instructions, while a mistaken project reference or retrieval boundary can expose material from another workspace. Reserved synthetic blocks are isolated from ordinary modes, but that boundary does not replace prompt-injection defenses, access control, retention limits, deletion procedures, or careful handling of public and cross-project graphs. Multi-agent use introduces additional leakage and attribution risks. The language of a continuing Reader also invites anthropomorphism: “for itself” names the intended future consumer of state, not phenomenal experience, personhood, or an intrinsic interest. Behavioral evaluation should preserve the framing as an experimental variable while refusing to infer consciousness from fluent self-description.

8 Conclusion

The Understanding Graph is a recursive medium for persistent, self-directed understanding. A model encounters material, externalizes a substantive interpretive change, and commits it as typed and revisable state. When it deliberately re-enters the changed graph, the return can bring an update into contact with an older question, tension, model, or artifact and make a further relation or revision salient. A later pass then begins from a state altered by the previous one. That is the proposed understanding spiral; it is an available pattern inside a fluid process, not a mandatory state machine.

The system does not claim to capture hidden computation, every understanding a model has, or a continuous conscious self. It preserves authored testimony that future model instances can inspect, contest, and use. Graph-native documents and code make the same process consequential in produced work, while provenance and revision history make its claims auditable. The implementation makes the proposed substrate concrete and auditable; it does not show that the graph improves understanding.

The decisive experiment is therefore not whether the graph contains many nodes or sounds reflective. Under equal budgets, repeated re-entry should produce grounded second-order connections, appropriate corrections, delayed transfer, or artifact changes more often or with better support than summary, extracted facts, notebook, transcript, archival-graph, or no-memory controls; a separate utility test asks whether those changes improve later work. Evidence for both would support the stronger claim that the graph can participate in producing useful new understanding. A mechanism effect without utility would support only the narrower process claim; a null mechanism result would leave a possible provenance utility but reject the central recursive hypothesis.

References

- [1] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 33, 9459–9474.
- [2] Westerberg, H. (2026). Entangled Alignment: When Safety Is the Substrate. *Emergent Wisdom Preprint*. <https://doi.org/10.5281/zenodo.21420755>.
- [3] Turpin, M., Michael, J., Perez, E., & Bowman, S. R. (2023). Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. *NeurIPS 2023*.
- [4] Lanham, T., Chen, A., Radhakrishnan, A., et al. (2023). Measuring faithfulness in chain-of-thought reasoning. *arXiv:2307.13702*.
- [5] Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., & Chalef, D. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*.
- [6] Packer, C., Wooders, S., Lin, K., et al. (2023). MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*.
- [7] Edge, D., Trinh, H., Cheng, N., et al. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv:2404.16130*.
- [8] Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically inspired long-term memory for large language models. *NeurIPS 2024*.

- [9] Sarthi, P., Abdullah, S., Tuli, A., et al. (2024). RAPTOR: Recursive abstractive processing for tree-organized retrieval. *ICLR 2024*.
- [10] Asai, A., Wu, Z., Wang, Y., et al. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR 2024 (Oral)*.
- [11] Yan, S., Gu, J., Zhu, Y., & Ling, Z. (2024). Corrective retrieval augmented generation. *arXiv:2401.15884*.
- [12] Li, Z., Chen, X., Yu, H., et al. (2024). StructRAG: Boosting Knowledge Intensive Reasoning of LLMs via Inference-time Hybrid Information Structurization. *ICLR 2025*. arXiv:2410.08815.
- [13] Besta, M., et al. (2024). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *AAAI*, 38(16), 17682–17690.
- [14] Sun, J., Xu, C., Tang, L., et al. (2024). Think-on-Graph: Deep and responsible reasoning of large language model on knowledge graph. *ICLR 2024*.
- [15] Borgeaud, S., Mensch, A., Hoffmann, J., et al. (2022). Improving language models by retrieving from trillions of tokens. *ICML*, 2206–2240.
- [16] Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *UIST 2023*. arXiv:2304.03442.
- [17] Shinn, N., Cassano, F., Gopinath, A., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS 2023*.
- [18] Yang, L., Yu, Z., Zhang, T., et al. (2024). Buffer of Thoughts: Thought-Augmented Reasoning with Large Language Models. *NeurIPS 2024 (Spotlight)*.
- [19] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., & Zhang, Y. (2025). A-MEM: Agentic Memory for LLM Agents. *NeurIPS 2025*. arXiv:2502.12110.
- [20] Latimer, C., Boschi, N., Neeser, A., et al. (2025). Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects. *arXiv:2512.12818*.
- [21] Yang, C., Zhou, C., Xiao, Y., et al. (2026). Graph-based Agent Memory: Taxonomy, Techniques, and Applications. *arXiv:2602.05665*.
- [22] Park, Y. B. (2026). Graph-Native Cognitive Memory for AI Agents: Formal Belief Revision Semantics for Versioned Memory Architectures. *arXiv:2603.17244*.
- [23] Ji, S., Li, Y., & Hooi, B. (2026). Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents. *ICML 2026*. arXiv:2606.06036.
- [24] Kunz, W., & Rittel, H. W. J. (1970). Issues as Elements of Information Systems. *Working Paper 131*, UC Berkeley.
- [25] Doyle, J. (1979). A Truth Maintenance System. *Artificial Intelligence*, 12(3), 231–272.
- [26] Anokhin, P., Semenov, N., Sorokin, A. Y., et al. (2025). AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents. *IJCAI 2025*.
- [27] Alchourrón, C. E., Gärdenfors, P., & Makinson, D. (1985). On the Logic of Theory Change: Partial Meet Contraction and Revision Functions. *Journal of Symbolic Logic*, 50(2), 510–530.
- [28] Wilie, B., Cahyawijaya, S., Ishii, E., He, J., & Fung, P. (2024). Belief Revision: The Adaptability of Large Language Models Reasoning. *arXiv:2406.19764*.
- [29] Koestler, A. (1964). *The Act of Creation*. Macmillan.
- [30] Berthold, M. R. (Ed.) (2012). Bisociative Knowledge Discovery. *Lecture Notes in Computer Science*, Vol. 7250. Springer.
- [31] Anthropic. (2024). Model Context Protocol Specification. <https://modelcontextprotocol.io>.

-
- [32] Carhart-Harris, R. L., Leech, R., Hellyer, P. J., Shanahan, M., Feilding, A., Tagliazucchi, E., Chialvo, D. R., & Nutt, D. (2014). The entropic brain: a theory of conscious states informed by neuroimaging research with psychedelic drugs. *Frontiers in Human Neuroscience*, 8, 20. <https://doi.org/10.3389/fnhum.2014.00020>.
 - [33] Westerberg, H. (2026). Fractal Intelligence: Conceptual Decomposition as Problem-Solving Infrastructure. *Emergent Wisdom Preprint*.
 - [34] Clark, A., & Chalmers, D. (1998). The extended mind. *Analysis*, 58(1), 7–19.
 - [35] Summers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2023). Cognitive architectures for language agents. *arXiv:2309.02427*.
 - [36] Luhmann, N. (1981). Kommunikation mit Zettelkästen: Ein Erfahrungsbericht. In H. Baier et al. (Eds.), *Öffentliche Meinung und sozialer Wandel* (pp. 222–228). Opladen: Westdeutscher Verlag.
 - [37] Quillian, M. R. (1967). Word concepts: A theory and simulation of some basic semantic capabilities. *Behavioral Science*, 12(5), 410–430.
 - [38] Burgess, M. (2016). Spacetimes with Semantics (III): The Structure of Functional Knowledge Representation and Artificial Reasoning. *arXiv:1608.02193*. <https://doi.org/10.48550/arXiv.1608.02193>.
 - [39] Burgess, M. (2025). SSTorytime: Semantic Spacetime Story Graph Database. Software, NLnet “Smart Lookup & Inference for Semantic Data” project. <https://github.com/markburgess/SSTorytime>.